Palm OS Programming Recipes

Accessing expansion media with the VFS and Expansion Managers

Conduits can access and manipulate information on expansion media using the VFS and Expansion Managers in the Palm Conduit Development Kit. This recipe will explain how to detect expansion media, in this case card(s), and retrieve information pertaining to the expansion media. The Virtual File System (VFS) Manager provides a robust interface for data access on expansion technologies. The Expansion Manager is a software layer that manages slot drivers on Palm Powered handhelds. On the handheld device, the Expansion Manager will broadcast notifications of card insertion/removal, play alerts for card insertion/removal, and mount or unmount card-resident volumes. This recipe will focus on using VFS Manager functions to obtain the number of accessible volumes on a handheld and proceed to obtain information pertaining to the found volumes.

Step 1: Determining expansion capabilities of the device

The following code will determine if the Expansion Manager is present on the handheld. If the `VFSSupport` function returns a zero value for either of it's parameters, the developer should return appropriate error codes to the caller. This code verifies both parameters and returns generic error codes as an example.

```
// TODO - define appropriate error codes
#define RECIPE_ERR_NO_SLOT              0
#define RECIPE_ERR_NO_VOLUMES           0
#define CARDINFO_TEXT                   "  - Card Information\n
        * Product: %s\n    * Manufacturer: %s\n
                * Type: %s\n    * ID: %s"

// Begin VFS Recipe Code. Declare all data required for Steps 1-3

DWORD dwExpansionMgrVersion;
DWORD dwVolumesAvailable;
ExpCardInfoType tCardInfo;      // ExpCardInfoType structure.
WORD *pwVolRefNumList;
WORD wNumVolumes = 0;
VolumeInfoType tVolumeInfo;     // VolumeInfoType structure.

// Determine whether the Expansion Manager is present on the handheld
if (retval = VFSSupport(dwExpansionMgrVersion,dwVolumesAvailable))
{
        // VFSSupport returned an error code
        return (retval);
}

if (dwExpansionMgrVersion==0)
{
        // No expansion slot is present (return appropriate error code)
        return (RECIPE_ERR_NO_SLOT);
}

if (dwVolumesAvailable==0)
{
        // No file system on the card OR no card in the slot
```

```
        // (return appropriate error code)
        return (RECIPE_ERR_NO_VOLUMES);
}
```

Step 2: Obtaining the number of volumes and volume reference numbers

The following code will determine the number of available volumes. The first call to `VFSVolumeEnumerate` will return the number of mounted volumes. If the number of volumes is determined to be one or more, proceed to allocate a buffer for volume reference numbers. The second call to `VFSVolumeEnumerate` will return the volume reference numbers in the allocated buffer.

```
// **NOTE** This code begins immediately follows Step 1 (see above)
// Query the device for the number of volumes.
// The first call returns only the number of mounted volumes,
// not their reference numbers.
if (VFSVolumeEnumerate (&wNumVolumes, NULL))
{
        // VFSVolumeEnumerate returned an error
        return (retval);
}

if (wNumVolumes>0)
{
        // Allocate buffer for volume reference list.
        pwVolRefNumList = new WORD [wNumVolumes];

        if (pwVolRefNumList != NULL)
        {
                // Get the volume reference numbers.
                if (retval = VFSVolumeEnumerate(&wNumVolumes,
                        pwVolRefNumList))
                {
                        // VFSVolumeEnumerate returned an error
                        return (retval);
                }
        }
```

Step 3: Obtaining volume information and card information

The following code will iterate on the number of volumes obtained on the handheld. For each volume we will determine if the volume is present on an expansion slot. If so, check for the presence of an expansion card and obtain information for the card. This code completes the procedure begun in the previous two steps and will log card information to the HotSync Log:

```
        // **NOTE** This code begins immediately follows Step 2 (see above)
        // Iterate through the volumes
        for (int nVolume=0; nVolume<wNumVolumes; nVolume++)
        {
                if (retval = VFSVolumeInfo(pwVolRefNumList[nVolume],
                        &tVolumeInfo))
                {
                        // VFSVolumeInfo returned an error code
                        return (retval);
                }

                // the slotRefNum is only valid when the
```

```
                //mountClass is vfsMountClass_SlotDriver
                if ((tVolumeInfo.mountClass) ==
                        vfsMountClass_SlotDriver)
                {
                        // Check for a card in the specified slot
                        if (ExpCardPresent((tVolumeInfo.slotRefNum),
                                NULL) == SYNCERR_NONE)
                        {
                                // Obtain information about
                                // the expansion card
                                if (ExpCardInfo((tVolumeInfo.slotRefNum),
                                        &tCardInfo,NULL) != SYNCERR_NONE)
                                {
                                        // ExpCardInfo returned an error
                                        return (retval);
                                }

                                // A card was located and queried for
                                // information
                                else
                                {
                                        // Write the card information
                                        // to the HotSync log
                                        LogAddFormattedEntry(slText,false,
                                        CARDINFO_TEXT,
                                        tCardInfo.productStr,
                                        tCardInfo.manufacturerStr,
                                        tCardInfo.deviceClassStr,
                                        tCardInfo.deviceUniqueIDStr);
                                }

                        } // end: ExpCardPresent condition

                } // end: vfsMountClass_SlotDriver condition

        } // end: iteration on the number of volumes

} // end: (volumes > 0 condition)
```

The procedure followed in this code will allow a developer to easily access expansion media on the handheld device. This examples was specific to expansion cards for the purposes of an example. The procedure followed in the code is generic and can be used or modified to satisfy the requirements of a conduit with expansion capabilites. A conduit developer can proceed to perform operations on the expansion media using the VFS Manager and Expansion Manager APIs provided in the Palm Conduit Development Kit and introduced in this recipe. **Complete Code sample (steps 1 - 3):**

```
// TODO - define appropriate error codes
#define RECIPE_ERR_NO_SLOT              0
#define RECIPE_ERR_NO_VOLUMES           0
#define CARDINFO_TEXT                   "  - Card Information\n
        * Product: %s\n
        * Manufacturer: %s\n
        * Type: %s\n    * ID: %s"

// Begin VFS Recipe Code

DWORD dwExpansionMgrVersion;
```

```
DWORD dwVolumesAvailable;
ExpCardInfoType tCardInfo;      // ExpCardInfoType structure.
WORD *pwVolRefNumList;
WORD wNumVolumes = 0;
VolumeInfoType tVolumeInfo;     // VolumeInfoType structure.

// Determine whether the Expansion Manager is
// present on the handheld
if (retval = VFSSupport(dwExpansionMgrVersion,
        dwVolumesAvailable))
{
        // VFSSupport returned an error code
        return (retval);
}

if (dwExpansionMgrVersion==0)
{
        // No expansion slot is present
        // (return appropriate error code)
        return (RECIPE_ERR_NO_SLOT);
}

if (dwVolumesAvailable==0)
{
        // No file system on the card OR no card in the
        // slot (return appropriate error code)
        return (RECIPE_ERR_NO_VOLUMES);
}

// Query the device for the number of volumes.
// The first call returns only the number of mounted
// volumes, not their reference numbers.
if (VFSVolumeEnumerate (&wNumVolumes, NULL))
{
        // VFSVolumeEnumerate returned an error
        return (retval);
}

if (wNumVolumes>0)
{
        // Allocate buffer for volume reference list.
        pwVolRefNumList = new WORD [wNumVolumes];

        if (pwVolRefNumList != NULL)
        {
                // Get the volume reference numbers.
                if (retval = VFSVolumeEnumerate(&wNumVolumes,
                        pwVolRefNumList))
                {
                        // VFSVolumeEnumerate returned an error
                        return (retval);
                }
        }

        // Iterate through the volumes
        for (int nVolume=0; nVolume<wNumVolumes; nVolume++)
        {
                if (retval = VFSVolumeInfo(pwVolRefNumList[nVolume],
                        &tVolumeInfo))
                {
```

```
                        // VFSVolumeInfo returned an error code
                        return (retval);
                }

                // the slotRefNum is only valid when the mountClass is
                // vfsMountClass_SlotDriver
                if ((tVolumeInfo.mountClass) ==
                        vfsMountClass_SlotDriver)
                {
                        // Check for a card in the specified slot
                        if (ExpCardPresent((tVolumeInfo.slotRefNum),
                                NULL) == SYNCERR_NONE)
                        {
                                // Obtain information
                                // about the expansion card
                                if (ExpCardInfo((tVolumeInfo.slotRefNum),
                                        &tCardInfo,NULL) != SYNCERR_NONE)
                                {
                                        // ExpCardInfo returned an error
                                        return (retval);
                                }

                                // A card was located and queried
                                // for information
                                else
                                {
                                        // Write the card information
                                        // to the HotSync log
                                        LogAddFormattedEntry(slText,false,
                                        CARDINFO_TEXT,
                                        tCardInfo.productStr,
                                        tCardInfo.manufacturerStr,
                                        tCardInfo.deviceClassStr,
                                        tCardInfo.deviceUniqueIDStr);
                                }

                        } // end: ExpCardPresent condition

                } // end: vfsMountClass_SlotDriver condition

        } // end: iteration on the number of volumes

} // end: (volumes > 0 condition)
```